



المحاضرة الثامنة

المؤشرات

Pointers

Topics for Lecture 8
8.1 Getting the Address of a Variable
8.2 Pointer Variables
8.3 The Relationship between Arrays and Pointers
8.4 Pointer Arithmetic
8.5 Initializing Pointers
8.6 Comparing Pointers
8.7 Pointers as Function Parameters
8.8 Dynamic Memory Allocation
8.9 Returning Pointers from Functions

8.1 Getting the Address of a Variable

المفهوم: يُرجع مُعامل العنوان (&) عنوان ذاكرة المتغير.

يُخصص لكل متغير مساحة ذاكرة كافية لاحتواء قيمة من نوع بياناته. على سبيل المثال، في أجهزة الكمبيوتر، من الشائع تخصيص بايت واحد للأحرف (char)، وبايتين للأعداد القصيرة (short)، و4 بايتات للأعداد الصحيحة والطويلة والعائمة (int, long, and float)، و8 بايتات للأعداد المضاعفة (double).

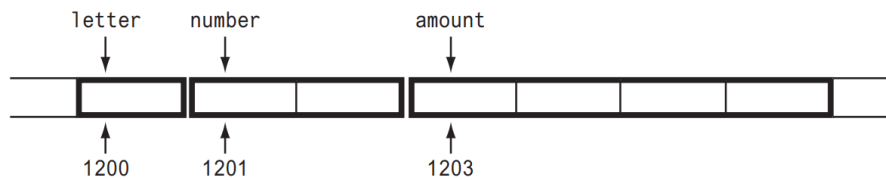
لكل بايت من الذاكرة عنوان فريد (unique address). عنوان المتغير هو عنوان أول بايت مُخصص له. لنفترض أن المتغيرات التالية مُعرّفة في برنامج:

char letter;

short number;

float amount;

Figure 9-1 Variables and their addresses



In Figure 9-1, the variable `letter` is shown at address 1200, `number` is at address 1201, and `amount` is at address 1203.



NOTE: The addresses of the variables shown in Figure 9-1 are arbitrary values used only for illustration purposes.

يتم الحصول على عنوان متغير باستخدام مُعامل في لغة C++. عند وضع مُعامل العنوان (&) قبل اسم المتغير، يُرجع عنوانه. فيما يلي تعبير يُرجع عنوان المتغير `amount`:

&amount

وهنا بيان يعرض عنوان المتغير على الشاشة:

cout << &amount;



NOTE: Do not confuse the address operator with the & symbol used when defining a reference variable.

يوضح البرنامج 9-1 استخدام عامل العنوان لعرض عنوان وحجم ومحتويات المتغير.

Program 9-1

```
1 // This program uses the & operator to determine a variable's
2 // address and the sizeof operator to determine its size.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     int x = 25;
9
10    cout << "The address of x is " << &x << endl;
11    cout << "The size of x is " << sizeof(x) << " bytes\n";
12    cout << "The value in x is " << x << endl;
13    return 0;
14 }
```

Program Output

```
The address of x is 0x8f05
The size of x is 4 bytes
The value in x is 25
```



NOTE: The address of the variable x is displayed in hexadecimal. This is the way addresses are normally shown in C++.

8.2 Pointer Variables

المفهوم: صُممت متغيرات المؤشر (*Pointer variables*)، والتي تُسمى عادةً مؤشرات (*pointers*) فقط، لحفظ عناوين الذاكرة. باستخدام متغيرات المؤشر، يمكنك معالجة البيانات المخزنة في متغيرات أخرى بشكل غير مباشر.

متغير المؤشر، والذي يُسمى عادةً مؤشرًا فقط، هو متغير خاص يحفظ عنوان ذاكرة. وكما صُممت متغيرات `int` لحفظ الأعداد الصحيحة، وصُممت متغيرات `double` لحفظ الأعداد العشرية، صُممت متغيرات المؤشر لحفظ عناوين الذاكرة.

تُحدد عناوين الذاكرة مواقع محددة في ذاكرة الحاسوب. ولأن متغير المؤشر يحمل عنوان ذاكرة، يُمكن استخدامه لحفظ موقع بيانات أخرى. هذا يُعطيك فكرة عن سبب تسميته بمؤشر: فهو "يشير" إلى بيانات مخزنة في ذاكرة الحاسوب. كما تتيح لك متغيرات المؤشر العمل مع البيانات التي تُشير إليها.

لقد استخدمنا عناوين الذاكرة للعمل مع البيانات. تذكر أنه عند تمرير مصفوفة كمعامل إلى دالة، فإننا في الواقع نمرر عنوان بداية المصفوفة. على سبيل المثال، لنفترض أن لدينا مصفوفة باسم `numbers`، ونستدعي دالة `showValues` كما هو موضح هنا:

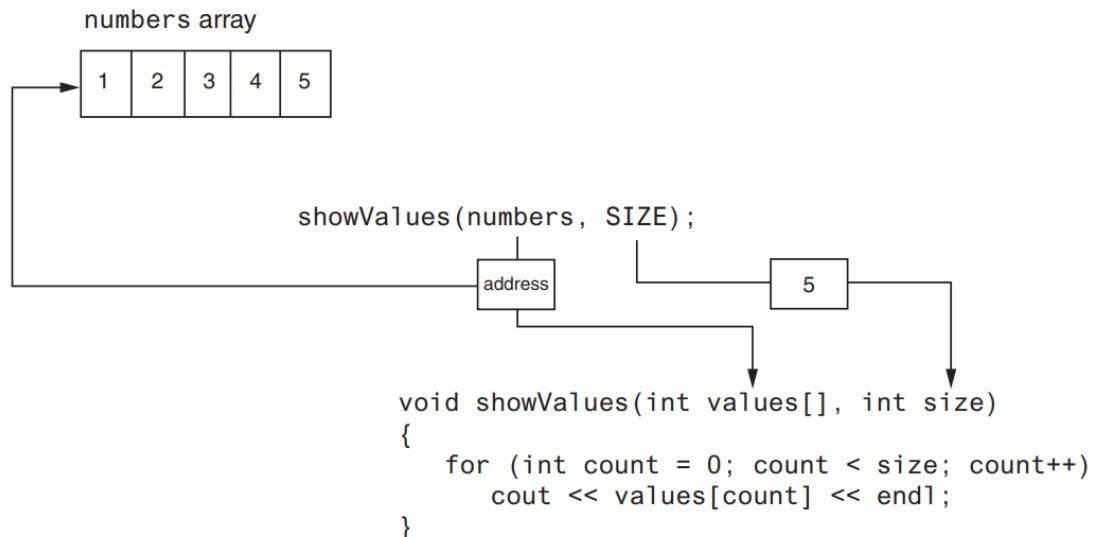
```
const int SIZE = 5;
int numbers[SIZE] = { 1, 2, 3, 4, 5 };
showValues(numbers, SIZE);
```

في هذا الكود، نمرر اسم المصفوفة (`numbers`) وحجمها (`SIZE`) كوسائط إلى دالة `showValues`. إليك تعريف دالة `showValues`:

```
void showValues(int values[], int size)
{
    for (int count = 0; count < size; count++)
        cout << values[count] << endl;
}
```

في هذه الدالة، يستقبل مُعامل القيم عنوان مصفوفة الأرقام. ويعمل كمؤشر لأنه يُشير إلى مصفوفة الأرقام، كما هو موضح في الشكل 9-2.

Figure 9-2 Passing an array to a function



انتبه للشرح فيما يأتي للتمييز بين متغير المرجع (reference variable) والمؤشر (pointer): يعمل متغير المرجع كاسم مستعار (alias) لمتغير آخر. ويُسمى متغيرًا مرجعيًا لأنه يُشير إلى متغير آخر في البرنامج. أي إجراء تُجرّيه على متغير المرجع يُجرى في الواقع على المتغير الذي يُشير إليه. على سبيل المثال، لنفترض أن لدينا المتغير `jellyDonuts`، ونمرر هذا المتغير إلى دالة `getOrder`، كما هو موضح هنا:

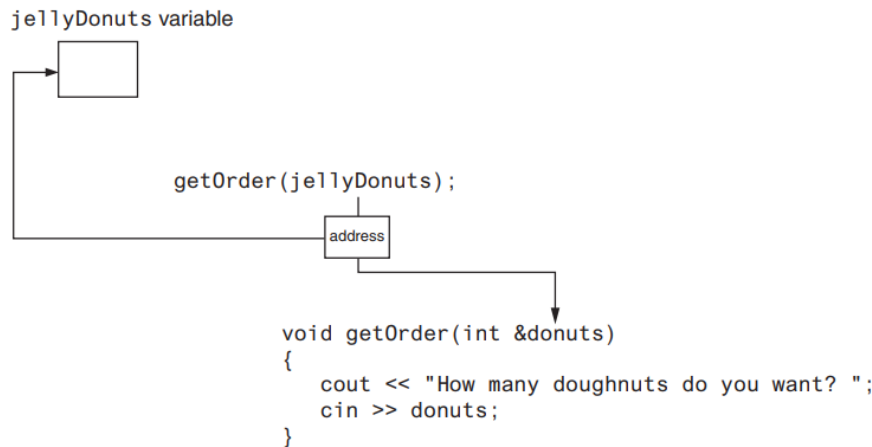
```
int jellyDonuts;
getOrder(jellyDonuts);
```

Here is the definition for the getOrder function:

```
void getOrder(int &donuts)
{
    cout << "How many doughnuts do you want? ";
    cin >> donuts;
}
```

في هذه الدالة، يُعدّ مُعامل الدونات مُتغيرًا مرجعيًا، ويستقبل عنوان مُتغير jellyDonuts. يعمل كمؤشر لأنه يُشير إلى مُتغير jellyDonuts كما هو مُوضح في الشكل 9-3.

Figure 9-3 Passing an argument by reference



داخل دالة getOrder، يُحيل مُعامل donuts إلى مُتغير jellyDonuts. أي إجراء على مُعامل donuts يُجرى في الواقع على مُتغير jellyDonuts. عندما يُدخل المستخدم قيمة، تستخدم جملة cin مُتغير donuts المرجعي لتخزين القيمة بشكل غير مباشر في مُتغير jellyDonuts. لاحظ أن الاتصال بين مُتغير donuts المرجعي ومُعامل jellyDonuts يُنشأ تلقائيًا بواسطة C++ عند استدعاء الدالة. عند كتابة هذا الكود، لن تحتاج إلى عناء البحث عن عنوان ذاكرة مُتغير jellyDonuts ثم تخزينه بشكل صحيح في مُتغير donuts المرجعي. عند تخزين قيمة في مُتغير donuts، لست مضطرًا لتحديد أن القيمة يجب تخزينها فعليًا في مُتغير jellyDonuts. C++ تتولى كل ذلك تلقائيًا.

في لغة C++، تُعدّ متغيرات المؤشر (pointers) آلية أخرى لاستخدام عناوين الذاكرة للتعامل مع البيانات. تُشبه متغيرات المؤشر متغيرات المرجع، إلا أنها تعمل بمستوى أدنى. أعني بذلك أن لغة C++ لا تُنجز تلقائيًا نفس العمل مع متغيرات المؤشر كما تفعل مع متغيرات المرجع. لجعل متغير

مؤشر يُشير إلى عنصر آخر في الذاكرة، يجب عليك كتابة شيفرة برمجية تُحضر عنوان ذاكرة ذلك العنصر وتُعيّنه لمتغير المؤشر. أيضًا، عند استخدام متغير مؤشر لتخزين قيمة في موقع الذاكرة الذي يُشير إليه المؤشر، يجب أن تُحدد شيفرتك البرمجية أنه يجب تخزين القيمة في الموقع الذي يُشير إليه متغير المؤشر، وليس في متغير المؤشر نفسه.

نظرًا لسهولة التعامل مع متغيرات المرجع، فقد تتساءل عن سبب استخدامك للمؤشرات أصلًا. في لغة C++، تُعدّ المؤشرات مفيدة، بل ضرورية، للعديد من العمليات. إحدى هذه العمليات هي تخصيص الذاكرة الديناميكي. عند كتابة برنامج يتطلب العمل مع كمية غير معروفة من البيانات، يتيح لك تخصيص الذاكرة الديناميكي إنشاء متغيرات ومصفوفات وهياكل بيانات أكثر تعقيدًا في الذاكرة أثناء تشغيل البرنامج. سنناقش تخصيص الذاكرة الديناميكي بمزيد من التفصيل في هذه الحاضرة. تُعدّ المؤشرات مفيدة جدًا أيضًا في الخوارزميات التي تتعامل مع المصفوفات وتعمل مع أنواع معينة من السلاسل النصية. في البرمجة كائنية التوجه، والتي ستتعلم عنها في محاضرات لاحقة، تُعدّ المؤشرات مفيدة جدًا لإنشاء الكائنات والعمل معها، ولمشاركة الوصول إليها.

Creating and Using Pointer Variables

التعريف	القراءة	التوضيح
<code>int *ptr;</code>	"ptr is a pointer to an int."	In this definition, the word int does not mean ptr is an integer variable.
<code>int* ptr;</code>		It means ptr can hold the address of an integer variable. Remember, pointers only hold one kind of value: an address.

في إصدار C++ 11، أُضيفت الكلمة المفتاحية `nullptr` لتمثيل العنوان 0. لذا، فإن تعيين `nullptr` لمتغير مؤشر يجعل المتغير يشير إلى العنوان 0. عند تعيين مؤشر إلى العنوان 0، يُشار إليه بمؤشر فارغ لأنه يشير إلى "لا شيء". إليك مثال لكيفية تعريف متغير مؤشر وتهيئته بالقيمة `nullptr`:

```
int *ptr = nullptr;
```



NOTE: If you are using an older compiler that does not support the C++ 11 standard, you should initialize pointers with the integer 0, or the value `NULL`. The value `NULL` is defined in the `<iostream>` header file (as well as other header files) to represent the value 0.

يوضح البرنامج 2-9 استخدامًا بسيطًا للغاية للمؤشر: تخزين وطباعة عنوان متغير آخر.

Program 9-2

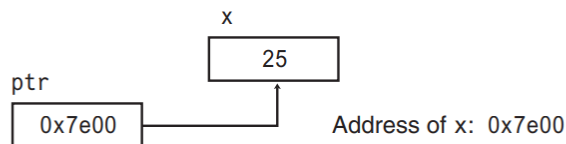
```
1 // This program stores the address of a variable in a pointer.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int x = 25;           // int variable
8     int *ptr = nullptr;   // Pointer variable, can point to an int
9
10    ptr = &x;              // Store the address of x in ptr
11    cout << "The value in x is " << x << endl;
12    cout << "The address of x is " << ptr << endl;
13    return 0;
14 }
```

Program Output

The value in x is 25
The address of x is 0x7e00

في البرنامج 9-2، تم تعريف متغيرين: `x` و `ptr`. المتغير `x` هو عدد صحيح، والمتغير `ptr` هو مؤشر إلى عدد صحيح. تم تهيئة المتغير `x` بالقيمة 25، وتم تهيئة المتغير `ptr` بالقيمة `nullptr`. بعد ذلك، تم تعيين عنوان `x` للمتغير `ptr` باستخدام العبارة في السطر 10.

Figure 9-4 `ptr` points to `x`



As shown in Figure 9-4, `x`, which is located at memory address `0x7e00`, contains the number 25. The `ptr` pointer contains the address `0x7e00`. In essence, it “points” to the variable `x`.

الفائدة الحقيقية للمؤشرات هي أنها تتيح لك الوصول غير المباشر إلى المتغير المُشار إليه وتعديله. في البرنامج 9-2، على سبيل المثال، يمكن استخدام `ptr` لتغيير محتوى المتغير `x`. يتم ذلك باستخدام مُعامل الإحالة (*indirection operator*)، وهو علامة النجمة (*). عند وضع مُعامل الإحالة أمام اسم متغير المؤشر، فإنه يُلغى مرجعية المؤشر. عند العمل مع مؤشر مُلغى المرجعية (dereferenced pointer)، فإنك تعمل في الواقع مع القيمة التي يُشير إليها المؤشر. هذا موضح في البرنامج 9-3.

Program 9-3

```
1 // This program demonstrates the use of the indirection operator.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     int x = 25;           // int variable
8     int *ptr = nullptr;   // Pointer variable, can point to an int
9
10    ptr = &x;             // Store the address of x in ptr
11
12    // Use both x and ptr to display the value in x.
13    cout << "Here is the value in x, printed twice:\n";
14    cout << x << endl;    // Displays the contents of x
15    cout << *ptr << endl; // Displays the contents of x
16
17    // Assign 100 to the location pointed to by ptr. This
18    // will actually assign 100 to x.
19    *ptr = 100;
```

Program 9-3 (continued)

```
20
21    // Use both x and ptr to display the value in x.
22    cout << "Once again, here is the value in x:\n";
23    cout << x << endl;    // Displays the contents of x
24    cout << *ptr << endl; // Displays the contents of x
25    return 0;
26 }
```

Program Output

```
Here is the value in x, printed twice:
25
25
Once again, here is the value in x:
100
100
```

يوضح البرنامج 9-4 أن المؤشرات يمكن أن تشير إلى متغيرات مختلفة.

Program 9-4

```
1 // This program demonstrates a pointer variable referencing
2 // different variables.
3 #include <iostream>
4 using namespace std;
```



```

5
6 int main()
7 {
8     int x = 25, y = 50, z = 75;    // Three int variables
9     int *ptr = nullptr;           // Pointer variable
10
11     // Display the contents of x, y, and z.
12     cout << "Here are the values of x, y, and z:\n";
13     cout << x << " " << y << " " << z << endl;
14
15     // Use the pointer to manipulate x, y, and z.
16
17     ptr = &x;    // Store the address of x in ptr.
18     *ptr += 100; // Add 100 to the value in x.
19
20     ptr = &y;    // Store the address of y in ptr.
21     *ptr += 100; // Add 100 to the value in y.
22
23     ptr = &z;    // Store the address of z in ptr.
24     *ptr += 100; // Add 100 to the value in z.
25
26     // Display the contents of x, y, and z.
27     cout << "Once again, here are the values of x, y, and z:\n";
28     cout << x << " " << y << " " << z << endl;
29     return 0;
30 }

```

Program Output

```

Here are the values of x, y, and z:
25 50 75
Once again, here are the values of x, y, and z:
125 150 175

```



NOTE: So far, you've seen three different uses of the asterisk in C++:

- As the multiplication operator, in statements such as
distance = speed * time;
- In the definition of a pointer variable, such as
int *ptr = nullptr;
- As the indirection operator, in statements such as
*ptr = 100;

8.3 The Relationship between Arrays and Pointers

المفهوم: يمكن استخدام أسماء المصفوفات كمؤشرات ثابتة، ويمكن استخدام المؤشرات كأسماء مصفوفات.

تعلمت أن اسم المصفوفة، بدون أقواس ومؤشر سفلي (subscript)، يُمثل في الواقع عنوان بداية المصفوفة. هذا يعني أن اسم المصفوفة هو في الواقع مؤشر. يوضح البرنامج 5-9 ذلك من خلال عرض اسم مصفوفة مُستخدم مع عامل الإحالة (indirection operator).

Program 9-5

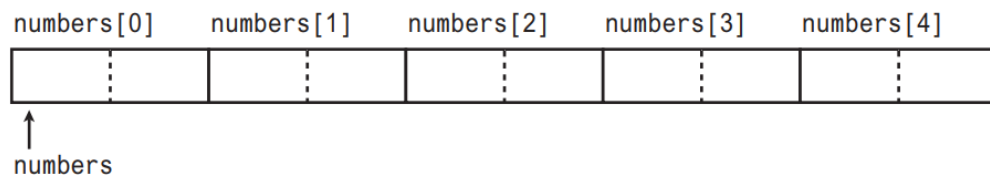
```
1 // This program shows an array name being dereferenced with the *
2 // operator.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     short numbers[] = {10, 20, 30, 40, 50};
9
10    cout << "The first element of the array is ";
11    cout << *numbers << endl;
12    return 0;
13 }
```

Program Output

The first element of the array is 10

لأن `numbers` تعمل كمؤشر لعنوان بداية المصفوفة، يتم استرداد العنصر الأول عند إلغاء الإشارة إلى `numbers`. فكيف يمكن استرداد محتويات المصفوفة بأكملها باستخدام عامل الإشارة غير المباشرة؟ تذكر أن عناصر المصفوفة يتم تخزينها معًا في الذاكرة، كما هو موضح في الشكل 5-9.

Figure 9-5 Array elements are stored in contiguous memory locations



من المنطقي أنه إذا كانت `numbers` هي عنوان `numbers[0]`، فيمكن إضافة قيم إلى الأرقام للحصول على عناوين العناصر الأخرى في المصفوفة. ومع ذلك، من المهم أن تعرف أن المؤشرات لا تعمل مثل المتغيرات العادية عند استخدامها في عبارات رياضية. في لغة C++، عندما تضيف قيمة إلى مؤشر، فإنك في الواقع تضيف تلك القيمة مضروبة في حجم نوع البيانات الذي يشير إليه المؤشر. بمعنى آخر، إذا أضفت واحدًا إلى `numbers`، فأنت في الواقع تضيف `sizeof(short) * 1` إلى

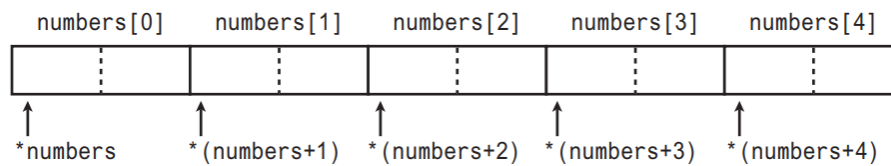
numbers. وإذا أضفت 2 إلى numbers، تكون النتيجة هي $\text{sizeof}(\text{short}) * 2 + \text{numbers}$ وهكذا دواليك. في نظام نموذجي، هذا يعني أن ما يلي صحيح، لأن الأعداد الصحيحة القصيرة (short) تستخدم عادةً 2 بايت:

```
*(numbers + 1) is actually *(numbers + 1 * 2)
*(numbers + 2) is actually *(numbers + 2 * 2)
*(numbers + 3) is actually *(numbers + 3 * 2)
```

and so forth.

هذا التحويل التلقائي يعني أنه يمكن استرداد عنصر في المصفوفة إما باستخدام الفهرس الخاص به، أو بإضافة الفهرس إلى مؤشر المصفوفة. إذا كان التعبير `*numbers` (وهو نفس `(numbers + 0)`) يسترجع العنصر الأول في المصفوفة، فإن `(numbers + 1)` يسترجع العنصر الثاني. وبالمثل، `(numbers + 2)` يسترجع العنصر الثالث، وهكذا دواليك. يوضح الشكل 9-6 التكافؤ بين تدوين الفهرس وتدوين المؤشر.

Figure 9-6 Subscript notation and pointer notation



NOTE: The parentheses are critical when adding values to pointers. The `*` operator has precedence over the `+` operator, so the expression `*number + 1` is not equivalent to `*(number + 1)`. `*number + 1` adds one to the contents of the first element of the array, while `*(number + 1)` adds one to the address in `number`, then dereferences it.

يوضح البرنامج 9-6 كيفية الوصول إلى محتويات المصفوفة بالكامل باستخدام ترميز المؤشر.

Program 9-6

```
1 // This program processes an array using pointer notation.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     const int SIZE = 5; // Size of the array
8     int numbers[SIZE]; // Array of integers
9     int count;          // Counter variable
10
11     // Get values to store in the array.
12     // Use pointer notation instead of subscripts.
13     cout << "Enter " << SIZE << " numbers: ";
14     for (count = 0; count < SIZE; count++)
15         cin >> *(numbers + count);
16
17     // Display the values in the array.
18     // Use pointer notation instead of subscripts.
19     cout << "Here are the numbers you entered:\n";
20     for (count = 0; count < SIZE; count++)
21         cout << *(numbers + count) << " ";
22     cout << endl;
23     return 0;
24 }
```

Program 9-6

(continued)

Program Output with Example Input Shown in Bold

```
Enter 5 numbers: 5 10 15 20 25 
Here are the numbers you entered:
5 10 15 20 25
```

When working with arrays, remember the following rule:

`array[index]` is equivalent to `*(array + index)`



WARNING! Remember that C++ performs no bounds checking with arrays. When stepping through an array with a pointer, it's possible to give the pointer an address outside of the array.

لإثبات مدى قوة العلاقة بين أسماء المصفوفات والمؤشرات، انظر إلى البرنامج 9-7. فهو يعرف مصفوفة من نوع `double` ومؤشرًا من نفس النوع، حيث يتم تعيين عنوان بداية المصفوفة لهذا

المؤشر. ليس فقط يتم استخدام ترميز المؤشر مع اسم المصفوفة، ولكن أيضًا يتم استخدام ترميز الفهرس مع المؤشر نفسه!

Program 9-7

```
1 // This program uses subscript notation with a pointer variable and
2 // pointer notation with an array name.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int NUM_COINS = 5;
9     double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
10    double *doublePtr;    // Pointer to a double
11    int count;            // Array index
12
13    // Assign the address of the coins array to doublePtr.
14    doublePtr = coins;
15
16    // Display the contents of the coins array. Use subscripts
17    // with the pointer!
18    cout << "Here are the values in the coins array:\n";
19    for (count = 0; count < NUM_COINS; count++)
20        cout << doublePtr[count] << " ";
21
22    // Display the contents of the array again, but this time
23    // use pointer notation with the array name!
24    cout << "\nAnd here they are again:\n";
25    for (count = 0; count < NUM_COINS; count++)
26        cout << *(coins + count) << " ";
```

Program 9-7 (continued)

```
27     cout << endl;
28     return 0;
29 }
```

Program Output

```
Here are the values in the coins array:
0.05 0.1 0.25 0.5 1
And here they are again:
0.05 0.1 0.25 0.5 1
```

لاحظ أن عامل العناوين (&) لا يكون مطلوبًا عند تعيين عنوان المصفوفة إلى مؤشر. لأن اسم المصفوفة يمثل بالفعل عنوانًا، لذا فإن استخدام العامل & سيكون غير صحيح. ومع ذلك، يمكنك استخدام عامل العناوين للحصول على عنوان عنصر معين في المصفوفة. على سبيل المثال،

`&numbers[1]` يحصل على عنوان العنصر `numbers[1]`. يتم استخدام هذه التقنية في البرنامج 8-9.

Program 9-8

```
1 // This program uses the address of each element in the array.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     const int NUM_COINS = 5;
8     double coins[NUM_COINS] = {0.05, 0.1, 0.25, 0.5, 1.0};
9     double *doublePtr = nullptr;    // Pointer to a double
10    int count;                       // Array index
11
12    // Use the pointer to display the values in the array.
13    cout << "Here are the values in the coins array:\n";
14    for (count = 0; count < NUM_COINS; count++)
15    {
16        // Get the address of an array element.
17        doublePtr = &coins[count];
18
19        // Display the contents of the element.
20        cout << *doublePtr << " ";
21    }
22    cout << endl;
23    return 0;
24 }
```

Program Output

```
Here are the values in the coins array:
0.05 0.1 0.25 0.5 1
```

الفرق الوحيد بين أسماء المصفوفات ومتغيرات المؤشر هو أنه لا يمكنك تغيير العنوان الذي يشير إليه اسم المصفوفة. على سبيل المثال:

```
double readings[20], totals[20];
double *dptr = nullptr;
```

These statements are legal:

```
dptr = readings; // Make dptr point to readings.
dptr = totals;   // Make dptr point to totals.
```

But these are illegal:

```
readings = totals; // ILLEGAL! Cannot change readings.
totals = dptr;      // ILLEGAL! Cannot change totals.
```

Array names are *pointer constants*. You can't make them point to anything but the array they represent.

8.4 Pointer Arithmetic

المفهوم: يمكن إجراء بعض العمليات الحسابية على المؤشرات. يمكن تغيير محتويات متغيرات المؤشر باستخدام جمل رياضية تُجري عمليات جمع أو طرح. هذا موضح في البرنامج 9-9. الحلقة الأولى تزيد قيمة متغير المؤشر، وتمرره عبر كل عنصر من عناصر المصفوفة. الحلقة الثانية تُنقص قيمة المؤشر، وتمرره عكسيًا عبر المصفوفة.

Program 9-9

```
1 // This program uses a pointer to display the contents of an array.
2 #include <iostream>
3 using namespace std;
4
5 int main()
6 {
7     const int SIZE = 8;
8     int set[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
9     int *numPtr = nullptr; // Pointer
10    int count;              // Counter variable for loops
11
12    // Make numPtr point to the set array.
```

Program 9-9 (continued)

```
13    numPtr = set;
14
15    // Use the pointer to display the array contents.
16    cout << "The numbers in set are:\n";
17    for (count = 0; count < SIZE; count++)
18    {
19        cout << *numPtr << " ";
20        numPtr++;
21    }
22
23    // Display the array contents in reverse order.
24    cout << "\nThe numbers in set backward are:\n";
25    for (count = 0; count < SIZE; count++)
26    {
27        numPtr--;
28        cout << *numPtr << " ";
29    }
30    return 0;
31 }
```

Program Output

```
The numbers in set are:
5 10 15 20 25 30 35 40
The numbers in set backward are:
40 35 30 25 20 15 10 5
```



NOTE: Because numPtr is a pointer to an integer, the increment operator adds the size of one integer to numPtr, so it points to the next element in the array. Likewise, the decrement operator subtracts the size of one integer from the pointer.

- لا يمكن إجراء جميع العمليات الحسابية على المؤشرات. على سبيل المثال، لا يمكنك ضرب أو قسمة مؤشر. فيما يلي العمليات المسموح بها:
- يمكن استخدام معاملات ++ و -- لزيادة أو إنقاص متغير المؤشر
 - يمكن إضافة أو طرح عدد صحيح من متغير المؤشر. يمكن تنفيذ ذلك باستخدام معاملات + و -، أو معاملات += و -=
 - يمكن طرح مؤشر من مؤشر آخر.

8.5 Initializing Pointers

المفهوم: يمكن تهيئة المؤشرات بعنوان كائن موجود.

تذكر أن المؤشر مصمم للإشارة إلى كائن من نوع بيانات محدد. عند تهيئة مؤشر بعنوان، يجب أن يكون عنوان الكائن الذي يشير إليه المؤشر.

على سبيل المثال، التعريف التالي لـ `pint` صحيح لأن `myValue` عدد صحيح (`int`):

```
int myValue;
```

```
int *pint = &myValue;
```

ما يلي قانوني أيضًا لأن `ages` عبارة عن مصفوفة من الأعداد الصحيحة (`int`):

```
int ages[20];
```

```
int *pint = ages;
```

لكن التعريف التالي لـ `pint` غير قانوني لأن `myFloat` ليس `int`:

```
float myFloat;
```

```
int *pint = &myFloat; // Illegal!
```

يمكن تعريف المؤشرات في نفس العبارة مع متغيرات أخرى من نفس النوع. تُعرّف العبارة التالية متغيرًا صحيحًا، `myValue`، ثم تُعرّف مؤشرًا، `pint`، والذي يُهيأ بعنوان `myValue`:

```
int myValue, *pint = &myValue;
```

وُعرّف العبارة التالية مصفوفة `readings`، ومؤشرًا `marker`، يتم تهيئته بعنوان العنصر الأول في المصفوفة:

```
double readings[50], *marker = readings;
```

بالطبع، لا يمكن تهيئة المؤشر إلا بعنوان كائن مُعرّف مسبقًا. ما يلي غير قانوني، لأنه يتم تهيئة `pint` بعنوان كائن غير موجود بعد:

```
int *pint = &myValue; // Illegal!
```

```
int myValue;
```


8.6 Comparing Pointers

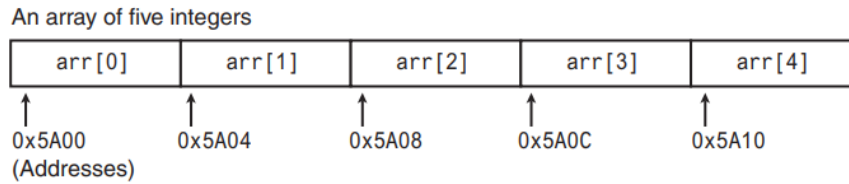
المفهوم: إذا جاء عنوان في الذاكرة قبل عنوان آخر، فإن العنوان الأول يعتبر "أقل من" الثاني. يمكن استخدام عوامل المقارنة في لغة C++ لمقارنة قيم المؤشرات.

يمكن مقارنة المؤشرات باستخدام أي من عوامل المقارنة في لغة C++:

عوامل المقارنة في C++			
النتيجة إذا كان $x = 5, y = 10$	مثال	المعنى	العامل
false	$x == y$	يساوي	<code>==</code>
true	$x != y$	لا يساوي	<code>!=</code>
false	$x > y$	أكبر من	<code>></code>
true	$x < y$	أقل من	<code><</code>
false	$x >= y$	أكبر من أو يساوي	<code>>=</code>
true	$x <= y$	أقل من أو يساوي	<code><=</code>

في المصفوفة، تُخزَّن جميع العناصر في مواقع ذاكرة متتالية، لذا يكون عنوان العنصر 1 أكبر من عنوان العنصر 0. ويوضح الشكل 7-9 ذلك.

Figure 9-7 Array elements



نظرًا لأن العناوين تصبح أكبر لكل عنصر لاحق في المصفوفة، فإن التعبيرات التي يتم اختبارها بواسطة عبارات الشرط التالية تكون جميعها صحيحة:

if (&arr[1] > &arr[0])

if (arr < &arr[4])

if (arr == &arr[0])

if (&arr[2] != &arr[3])



NOTE: Comparing two pointers is not the same as comparing the values the two pointers point to. For example, the following `if` statement compares the addresses stored in the pointer variables `ptr1` and `ptr2`:

```
if (ptr1 < ptr2)
```

The following statement, however, compares the values that `ptr1` and `ptr2` point to:

```
if (*ptr1 < *ptr2)
```

تتيح لك إمكانية مقارنة العناوين طريقة أخرى للتأكد من عدم تجاوز المؤشر لحدود المصفوفة. يُهيئ البرنامج 10-9 مؤشر ptr بعنوان بداية المصفوفة numbers. ثم يُمرر مؤشر ptr عبر المصفوفة numbers حتى يُصبح العنوان الذي يحتويه مساوياً لعنوان آخر عنصر فيها. ثم يُمرر المؤشر للخلف عبر المصفوفة حتى يُشير إلى العنصر الأول.

Program 9-10

```
1 // This program uses a pointer to display the contents
2 // of an integer array.
3 #include <iostream>
4 using namespace std;
5
6 int main()
7 {
8     const int SIZE = 8;
9     int numbers[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
10    int *ptr = numbers;    // Make ptr point to numbers
11
12    // Display the numbers in the array.
13    cout << "The numbers are:\n";
14    cout << *ptr << " ";    // Display first element
15    while (ptr < &numbers[SIZE - 1])
16    {
17        // Advance ptr to point to the next element.
18        ptr++;
19        // Display the value pointed to by ptr.
20        cout << *ptr << " ";
21    }
22
23    // Display the numbers in reverse order.
24    cout << "\nThe numbers backward are:\n";
25    cout << *ptr << " ";    // Display first element
26    while (ptr > numbers)
27    {
28        // Move backward to the previous element.
29        ptr--;
30        // Display the value pointed to by ptr.
31        cout << *ptr << " ";
32    }
33    return 0;
34 }
```

Program Output

```
The numbers are:
5 10 15 20 25 30 35 40
The numbers backward are:
40 35 30 25 20 15 10 5
```

8.7 Pointers as Function Parameters

المفهوم: يمكن استخدام المؤشر كمعامل دالة. فهو يُتيح للدالة الوصول إلى الوسيطة الأصلية، تمامًا كما يفعل عامل المرجع.

فيما يلي تعريف الدالة التي تستخدم المؤشر كوسيط:

```
void doubleValue(int *val)
{
    *val *= 2;
}
```

الغرض من هذه الدالة هو مضاعفة المتغير الذي يشير إليه `val`. بالطبع، عند استدعاء الدالة، يجب استخدام عنوان المتغير المراد مضاعفته كمعامل، وليس المتغير نفسه. إليك مثال على استدعاء دالة `doubleValue`:

```
doubleValue(&number);
```

تستخدم هذه العبارة مُعامل العنوان (&) لتمرير عنوان `number` إلى مُعامل `val`. بعد تنفيذ الدالة، يكون مُحتوى `number` قد ضرب في 2. تم توضيح استخدام هذه الدالة في البرنامج 9-11.

Program 9-11

```
1  // This program uses two functions that accept addresses of
2  // variables as arguments.
3  #include <iostream>
4  using namespace std;
5
6  // Function prototypes
7  void getNumber(int *);
8  void doubleValue(int *);
9
10 int main()
11 {
12     int number;
13
14     // Call getNumber and pass the address of number.
15     getNumber(&number);
16
17     // Call doubleValue and pass the address of number.
18     doubleValue(&number);
19
20     // Display the value in number.
21     cout << "That value doubled is " << number << endl;
22     return 0;
23 }
24
```

Program 9-11*(continued)*

```

25 //*****
26 // Definition of getNumber. The parameter, input, is a pointer. *
27 // This function asks the user for a number. The value entered *
28 // is stored in the variable pointed to by input. *
29 //*****
30
31 void getNumber(int *input)
32 {
33     cout << "Enter an integer number: ";
34     cin >> *input;
35 }
36
37 //*****
38 // Definition of doubleValue. The parameter, val, is a pointer. *
39 // This function multiplies the variable pointed to by val by *
40 // two. *
41 //*****
42
43 void doubleValue(int *val)
44 {
45     *val *= 2;
46 }

```

Program Output with Example Input Shown in Bold

Enter an integer number: **10**

That value doubled is 20

يمكن أيضًا استخدام متغيرات المؤشر لقبول عناوين المصفوفة كوسائط. ويمكن بعد ذلك استخدام ترميز الفهرسة أو ترميز المؤشر للتعامل مع محتويات المصفوفة. ويوضح البرنامج 9-12 ذلك.

Program 9-12

```

1 // This program demonstrates that a pointer may be used as a
2 // parameter to accept the address of an array.
3 #include <iostream>
4 #include <iomanip>
5 using namespace std;
6
7 // Function prototypes
8 void getSales(double *, int);
9 double totalSales(double *, int);
10
11 int main()
12 {
13     const int QTRS = 4;
14     double sales[QTRS];
15
16     // Get the sales data for all quarters.
17     getSales(sales, QTRS);
18
19     // Set the numeric output formatting.
20     cout << fixed << showpoint << setprecision(2);
21
22     // Display the total sales for the year.
23     cout << "The total sales for the year are $";
24     cout << totalSales(sales, QTRS) << endl;
25     return 0;
26 }
27

```

```

28 //*****
29 // Definition of getSales. This function uses a pointer to accept *
30 // the address of an array of doubles. The function asks the user *
31 // to enter sales figures and stores them in the array. *
32 //*****
33 void getSales(double *arr, int size)
34 {
35     for (int count = 0; count < size; count++)
36     {
37         cout << "Enter the sales figure for quarter ";
38         cout << (count + 1) << ": ";
39         cin >> arr[count];
40     }
41 }
42

```

Program 9-12 (continued)

```

43 //*****
44 // Definition of totalSales. This function uses a pointer to *
45 // accept the address of an array. The function returns the total *
46 // of the elements in the array. *
47 //*****
48 double totalSales(double *arr, int size)
49 {
50     double sum = 0.0;
51
52     for (int count = 0; count < size; count++)
53     {
54         sum += *arr;
55         arr++;
56     }
57     return sum;
58 }

```

Program Output with Example Input Shown in Bold

```

Enter the sales figure for quarter 1: 10263.98 
Enter the sales figure for quarter 2: 12369.69 
Enter the sales figure for quarter 3: 11542.13 
Enter the sales figure for quarter 4: 14792.06 
The total sales for the year are $48967.86

```

لاحظ في دالة `getSales` في البرنامج 9-12، أنه على الرغم من تعريف المعلمة `arr` كمؤشر، إلا أنه يتم استخدام تدوين المؤشر السفلي في جملة `cin` في السطر 39:

`cin >> arr[count];`

في دالة `totalSales`، يتم استخدام `arr` مع عامل الإحالة غير المباشرة (indirection operator) في السطر 54:

`sum += *arr;`

وفي السطر 55، يتم زيادة العنوان في arr ليشير إلى العنصر التالي:

arr++;



NOTE: The two previous statements could be combined into the following statement:

```
sum += *arr++;
```

The * operator will first dereference arr, then the ++ operator will increment the address in arr.

Pointers to Constants

لقد رأيت كيف يمكن تمرير عنوان عنصر إلى مُعامل مؤشر، وكيف يمكن استخدام المؤشر لتعديل العنصر المُمرَّر كمُعامل. في بعض الأحيان، يلزم تمرير عنوان عنصر ثابت إلى مؤشر. في هذه الحالة، يجب تعريف المؤشر كمؤشر إلى عنصر ثابت. على سبيل المثال، انظر إلى تعريف المصفوفة التالي:

```
const int SIZE = 6;
const double payRates[SIZE] = { 18.55, 17.45,
                                12.85, 14.97,
                                10.35, 18.89 };
```

في هذا الكود، payRates عبارة عن مصفوفة من الأعداد double الثابتة. هذا يعني أن كل عنصر في المصفوفة هو عدد double ثابت، ولن يسمح لنا المُجمِّع بكتابة كود يُغيِّر محتويات المصفوفة. إذا أردنا تمرير مصفوفة payRates إلى مُعامل مؤشر، فيجب تعريف المُعامل كمؤشر إلى مُعامل double ثابت. تُظهر الدالة التالية مثالاً على ذلك:

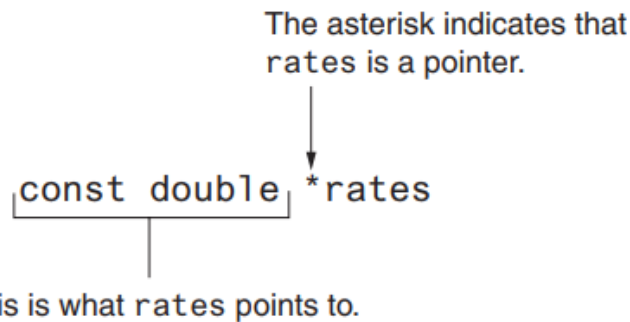
```
void displayPayRates(const double *rates, int size)
{
    // Set numeric output formatting.
    cout << setprecision(2) << fixed << showpoint;

    // Display all the pay rates.
    for (int count = 0; count < size; count++)
    {
        cout << "Pay rate for employee " << (count + 1)
              << " is $" << *(rates + count) << endl;
    }
}
```

في رأس الدالة، لاحظ أن معامل rates مُعرَّف كمؤشر إلى ثابت double.

تجدر الإشارة إلى أن كلمة ثابت تُطبَّق على ما يُشير إليه المعدل، وليس المعدل نفسه. كما هو موضح في الشكل 8-9.

Figure 9-8 rates is a pointer to const double



لأن rates مؤشر إلى ثابت، فلن يسمح لنا المُجمِّع (compiler) بكتابة شيفرة تُغيّر ما يُشير إليه rates. عند تمرير عنوان ثابت إلى مُتغيّر مؤشر، يجب تعريف المتغير كمؤشر إلى ثابت. لو حُذفت كلمة const من تعريف مُعامل rates، لكان خطأً في المُجمِّع (compiler).

Passing a Nonconstant Argument into a Pointer to a Constant

مع أن عنوان الثابت لا يُمرر إلا إلى مؤشر ثابت، إلا أن مؤشر ثابت يمكنه أيضاً استقبال عنوان عنصر غير ثابت. على سبيل المثال، انظر البرنامج 9-13.

Program 9-13

```
1 // This program demonstrates a pointer to const parameter
2 #include <iostream>
3 using namespace std;
4
5 void displayValues(const int *, int);
6
7 int main()
8 {
9     // Array sizes
10    const int SIZE = 6;
11
12    // Define an array of const ints.
13    const int array1[SIZE] = { 1, 2, 3, 4, 5, 6 };
14
15    // Define an array of nonconst ints.
16    int array2[SIZE] = { 2, 4, 6, 8, 10, 12 };
17
18    // Display the contents of the const array.
19    displayValues(array1, SIZE);
20
21    // Display the contents of the nonconst array.
22    displayValues(array2, SIZE);
23    return 0;
24 }
25
```

```

26 //*****
27 // The displayValues function uses a pointer to
28 // parameter to display the contents of an array.
29 //*****
30
31 void displayValues(const int *numbers, int size)
32 {
33     // Display all the values.
34     for (int count = 0; count < size; count++)
35     {
36         cout << *(numbers + count) << " ";
37     }
38     cout << endl;
39 }

```

Program Output

```

1 2 3 4 5 6
2 4 6 8 10 12

```



NOTE: When you are writing a function that uses a pointer parameter, and the function is not intended to change the data the parameter points to, it is always a good idea to make the parameter a pointer to `const`. Not only will this protect you from writing code in the function that accidentally changes the argument, but the function will be able to accept the addresses of both constant and nonconstant arguments.

Constant Pointers

في القسم السابق، ناقشنا مؤشرات البيانات الثابتة، أي المؤشرات التي تُشير إلى بيانات ثابتة. يمكنك أيضاً استخدام الكلمة المفتاحية `const` لتعريف مؤشر ثابت. إليك الفروق بين المؤشر إلى ثابت ومؤشر ثابت:

- يشير المؤشر إلى ثابت إلى عنصر ثابت. لا يمكن تغيير البيانات التي يشير إليها المؤشر، ولكن المؤشر نفسه يمكن أن يتغير.
 - في المؤشر ثابت، يكون المؤشر نفسه ثابتاً. بمجرد تهيئة المؤشر بعنوان، لا يمكنه الإشارة إلى أي شيء آخر.
- يُظهر الكود التالي مثلاً لمؤشر ثابت:

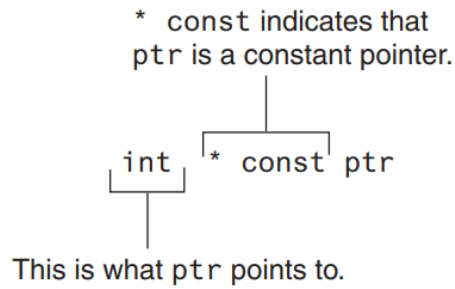
```

int value = 22;
int * const ptr = &value;

```

لاحظ في تعريف `ptr`، ظهور كلمة `const` بعد علامة النجمة. هذا يعني أن `ptr` مؤشر ثابت. يوضح الشكل 9-9 ذلك. في الكود، يُهيأ `ptr` بعنوان متغير `value`. ولأن `ptr` مؤشر ثابت، فسيحدث خطأ في المترجم إذا كتبنا كوداً يجعل `ptr` يشير إلى أي شيء آخر. مع ذلك، لن يحدث خطأ إذا استخدمنا `ptr` لتغيير محتوى القيمة. هذا لأن القيمة ليست ثابتة، و `ptr` ليس مؤشراً إلى `const`.

Figure 9-9 ptr is a const pointer to int



يجب تهيئة مؤشرات الثوابت بقيمة ابتدائية، كما هو موضح في المثال السابق. إذا استُخدم مؤشر ثابت كمعامل دالة، فسيتم تهيئة هذا العامل بالعنوان المُمرر إليه كمعامل، ولا يمكن تغييره للإشارة إلى أي شيء آخر أثناء تنفيذ الدالة. إليك مثال يحاول انتهاك هذه القاعدة:

```
void setToZero(int * const ptr)
{
    ptr = 0; // ERROR!! Cannot change the contents of ptr.
}
```

مُعامل هذه الدالة، ptr، هو مؤشر ثابت. لن يتم ترجمته (compiled) لأنه لا يُمكننا تضمين شيفرة في الدالة تُغير محتويات ptr. مع ذلك، لا يُشير ptr إلى ثابت، لذا يُمكننا تضمين شيفرة تُغير البيانات التي يُشير إليها ptr. إليك مثال على الدالة التي سيتم ترجمتها (compiled):

```
void setToZero(int * const ptr)
{
    *ptr = 0;
}
```

على الرغم من أن المعامل هو مؤشر ثابت، إلا أنه يُمكننا استدعاء الدالة عدة مرات باستخدام وسيطات مختلفة. سيمرر الكود التالي عناوين x و y و z بنجاح إلى دالة setToZero:

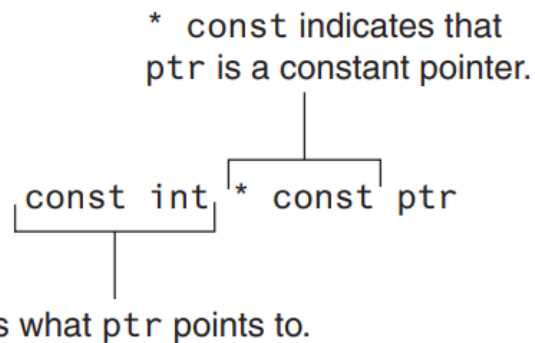
```
int x, y, z;
// Set x, y, and z to 0.
setToZero(&x);
setToZero(&y);
setToZero(&z);
```

Constant Pointers to Constants

على سبيل المثال، انظر إلى الكود التالي:

```
int value = 22;  
const int * const ptr = &value;
```

Figure 9-10 ptr is a const pointer to const int



في الكود، يُهيأ ptr بعنوان value. ولأن ptr مؤشر ثابت، لا يمكننا كتابة كود يجعل ptr يشير إلى أي شيء آخر. ولأن ptr مؤشر إلى ثابت، لا يمكننا استخدامه لتغيير محتوى value. يعرض الكود التالي مثالاً آخر على مؤشر ثابت إلى ثابت. هذه نسخة أخرى من دالة displayValues في البرنامج 9-13.

```
void displayValues(const int * const numbers, int size)  
{  
    // Display all the values.  
    for (int count = 0; count < size; count++)  
    {  
        cout << *(numbers + count) << " ";  
    }  
    cout << endl;  
}
```

8.8 Dynamic Memory Allocation

المفهوم: يمكن إنشاء المتغيرات وحذفها أثناء تشغيل البرنامج.

طالما أنك تعرف عدد المتغيرات التي ستحتاجها أثناء تنفيذ البرنامج، يمكنك تعريف هذه المتغيرات مسبقاً. على سبيل المثال، سيحتاج برنامج لحساب مساحة مستطيل إلى ثلاثة متغيرات: واحد لطول المستطيل، وآخر لعرضه، وآخر لحساب المساحة. إذا كنت تكتب برنامجاً لحساب رواتب 30 موظفاً، فمن المحتمل أن تُنشئ مصفوفة من 30 عنصراً لحساب مبلغ الراتب لكل شخص.

ولكن ماذا عن الأوقات التي لا تعرف فيها عدد المتغيرات التي تحتاجها؟ على سبيل المثال، لنفترض أنك تريد كتابة برنامج لحساب متوسط الاختبارات، والذي سيحسب متوسط أي عدد من الاختبارات. من الواضح أن البرنامج سيكون متعدد الاستخدامات، ولكن كيف يمكنك تخزين نتائج الاختبارات الفردية في الذاكرة إذا كنت لا تعرف عدد المتغيرات المطلوب تعريفها؟ ببساطة، يمكنك السماح للبرنامج بإنشاء متغيراته الخاصة "أثناء التشغيل" (on the fly). يُسمى هذا التخصيص الديناميكي للذاكرة، ولا يمكن تحقيقه إلا باستخدام المؤشرات. يعني التخصيص الديناميكي للذاكرة أن يطلب البرنامج، أثناء تشغيله، من الحاسوب تخصيص جزء من الذاكرة غير المستخدمة يكفي لاستيعاب متغير من نوع بيانات مُحدد. لنفترض أن برنامجاً يحتاج إلى إنشاء متغير عدد صحيح. سيُرسل طلباً إلى الحاسوب لتخصيص بايتات كافية لتخزين عدد صحيح. عندما يُلبي الحاسوب هذا الطلب، يجد ويخصص جزءاً من الذاكرة غير المستخدمة يكفي لاستيعاب المتغير. ثم يُعطي البرنامج عنوان بداية جزء الذاكرة. لا يُمكن للبرنامج الوصول إلى الذاكرة المُخصصة حديثاً إلا من خلال عنوانه، لذا يلزم وجود مؤشر لاستخدام تلك البايتات.

يطلب برنامج C++ ذاكرة مخصصة ديناميكياً من خلال عامل new. افترض أن البرنامج يحتوي على مؤشر إلى عدد صحيح مُعرّف كـ:

```
int *iptr = nullptr;
```

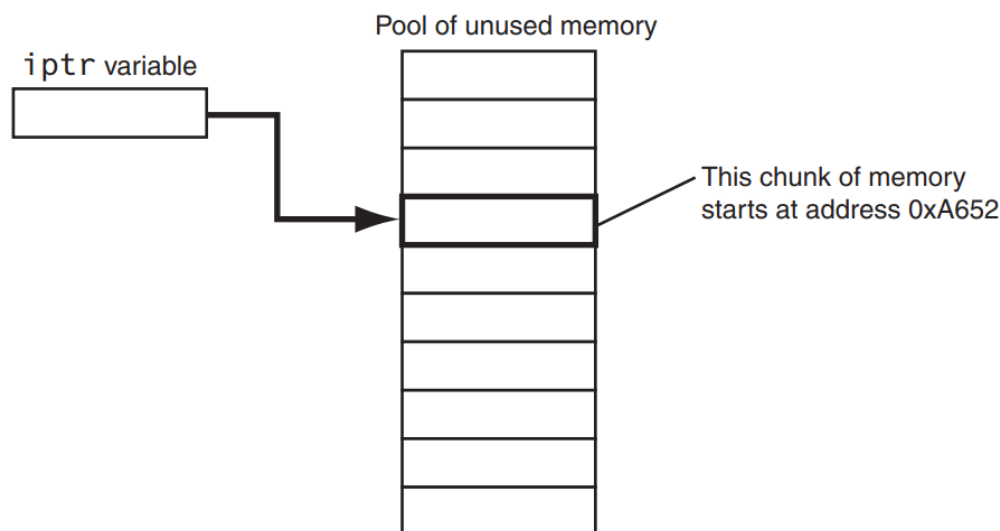
Here is an example of how this pointer may be used with the new operator:

```
iptr = new int;
```

تطلب هذه العبارة من الحاسوب تخصيص ذاكرة كافية لمتغير new int. مُعامل new هو نوع بيانات المتغير المنشأ. بمجرد تنفيذ العبارة، سيحتوي iptr على عنوان الذاكرة المُخصصة حديثاً. هذا موضح في الشكل 9-11. يمكن تخزين قيمة في هذا المتغير الجديد عن طريق إلغاء مرجع المؤشر (dereferencing the pointer):

```
*iptr = 25;
```

Figure 9-11 `iptr` points to a chunk of dynamically allocated memory



وفيما يلي بعض الأمثلة:

```
cout << *iptr;    // Display the contents of the new variable.
cin >> *iptr;     // Let the user input a value.
total += *iptr;   // Use the new variable in a computation.
```

على الرغم من أن العبارات أعلاه توضح استخدام عامل التشغيل `new`، إلا أن تخصيص متغير واحد ديناميكياً ليس له فائدة تُذكر. الاستخدام العملي لعامل التشغيل الجديد هو إنشاء مصفوفة ديناميكياً. فيما يلي مثال لكيفية تخصيص مصفوفة أعداد صحيحة مكونة من 100 عنصر:

```
iptr = new int[100];
```

بمجرد إنشاء المصفوفة، يُمكن استخدام المؤشر مع تدوين المؤشر السفلي (subscript notation) للوصول إليها. على سبيل المثال، يُمكن استخدام الحلقة التالية لتخزين القيمة 1 في كل عنصر:

```
for (int count = 0; count < 100; count++)
    iptr[count] = 1;
```

ولكن ماذا لو لم تكن هناك ذاكرة فارغة كافية لتلبية الطلب؟ ماذا لو طلب البرنامج كتلة كبيرة بما يكفي لاستيعاب مصفوفة من 100,000 عنصر من الأعداد العشرية (`float`)، ولم تكن هذه الكتلة من الذاكرة متاحة؟ عندما يتعذر تخصيص الذاكرة ديناميكياً، تُلقى لغة C++ استثناءً (*Throwing an exception*) وتُنتهي البرنامج. يعني طرح الاستثناء أن البرنامج يُشير إلى حدوث خطأ.

عند انتهاء البرنامج من استخدام جزء مخصص ديناميكياً من الذاكرة، يجب عليه تحريره للاستخدام المستقبلي. يُستخدم عامل الحذف (`delete`) لتحرير الذاكرة المخصصة باستخدام `new`. فيما يلي مثال على كيفية استخدام عامل الحذف لتحرير متغير واحد، يُشار إليه بواسطة `iptr`:

```
delete iptr;
```

إذا كان iptr يشير إلى مصفوفة مخصصة ديناميكيًا، فيجب وضع الرمز [] بين delete و iptr:

```
delete [] iptr;
```

قد يؤدي عدم تحرير الذاكرة المخصصة ديناميكيًا إلى تسرب ذاكرة (memory leak) في أحد البرامج. على سبيل المثال، لنفترض وجود دالة grabMemory التالية في أحد البرامج:

```
void grabMemory()
{
    const int SIZE = 100;
    // Allocate space for a 100-element
    // array of integers.
    int *iptr = new int[SIZE];
    // The function ends without deleting the memory!
}
```

لاحظ أن الدالة تُخصص ديناميكيًا مصفوفة عدد صحيح (int) مكونة من 100 عنصر، وتُعيّن عنوانها لمتغير المؤشر iptr. ولكن بعد ذلك، تنتهي الدالة دون حذف الذاكرة. بمجرد انتهاء الدالة، يختفي متغير المؤشر iptr. ونتيجةً لذلك، لا يحتوي البرنامج على مؤشر إلى الذاكرة المخصصة ديناميكيًا؛ وبالتالي، لا يمكن الوصول إلى الذاكرة أو حذفها. وستبقى غير مستخدمة طالما أن البرنامج قيد التشغيل.



WARNING! Only use pointers with delete that were previously used with new. If you use a pointer with delete that does not reference dynamically allocated memory, unexpected problems could result!

يوضح البرنامج 9-14 استخدام أمري "new" و "delete". يطلب البرنامج إدخال مبالغ المبيعات لأي عدد من الأيام. تُخزّن المبالغ في مصفوفة مخصصة ديناميكيًا، ثم تُجمع ويُحسب متوسطها.

Program 9-14

```
1 // This program totals and averages the sales amounts for any
2 // number of days. The amounts are stored in a dynamically
3 // allocated array.
4 #include <iostream>
5 #include <iomanip>
6 using namespace std;
7
8 int main()
9 {
10     double *sales = nullptr, // To dynamically allocate an array
11           total = 0.0,       // Accumulator
12           average;           // To hold average sales
13     int numDays,             // To hold the number of days of sales
14         count;               // Counter variable
15
16     // Get the number of days of sales.
17     cout << "How many days of sales amounts do you wish ";
18     cout << "to process? ";
19     cin >> numDays;
20
21     // Dynamically allocate an array large enough to hold
22     // that many days of sales amounts.
23     sales = new double[numDays];
24
25     // Get the sales amounts for each day.
26     cout << "Enter the sales amounts below.\n";
27     for (count = 0; count < numDays; count++)
28     {
29         cout << "Day " << (count + 1) << ": ";
30         cin >> sales[count];
31     }
32
33     // Calculate the total sales
34     for (count = 0; count < numDays; count++)
35     {
36         total += sales[count];
37     }
38
39     // Calculate the average sales per day
40     average = total / numDays;
41
42     // Display the results
43     cout << fixed << showpoint << setprecision(2);
44     cout << "\n\nTotal Sales: $" << total << endl;
45     cout << "Average Sales: $" << average << endl;
46
47     // Free dynamically allocated memory
48     delete [] sales;
49     sales = nullptr; // Make sales a null pointer.
50
51     return 0;
52 }
```

Program Output with Example Input Shown in Bold

How many days of sales amounts do you wish to process? **5**

Enter the sales amounts below.

Day 1: **898.63**

Day 2: **652.32**

Day 3: **741.85**

Day 4: **852.96**

Day 5: **921.37**

Total Sales: \$4067.13

Average Sales: \$813.43

8.9 Returning Pointers from Functions

المفهوم: يمكن للدوال (التوابع) إرجاع مؤشرات، ولكن يجب التأكد من أن العنصر الذي يشير إليه المؤشر لا يزال موجودًا.

كما هو الحال مع أي نوع بيانات آخر، قد تُرجع الدوال مؤشرات. على سبيل المثال، تُحدد الدالة التالية مُنهي العدم (null terminator) الذي يظهر في نهاية السلسلة (مثل حرف سلسلة نصية) وتُرجع مؤشرًا إليه:

```
char *findNull(char *str)
{
    char *ptr = str;

    while (*ptr != '\0')
        ptr++;
    return ptr;
}
```

يشير `char *` في رأس الدالة (header function) إلى أن الدالة تقوم بإرجاع مؤشر إلى `char`. عند كتابة دوال تُرجع مؤشرات، احرص على عدم إنشاء أخطاء يصعب اكتشافها. على سبيل المثال، حاول تحديد المشكلة في الدالة التالية:

```
string *getFullName()
{
    string fullName[3];
    cout << "Enter your first name: ";
    getline(cin, fullName[0]);
    cout << "Enter your middle name: ";
    getline(cin, fullName[1]);
    cout << "Enter your last name: ";
    getline(cin, fullName[2]);
    return fullName;
}
```

تكمّن المشكلة، بالطبع، في أن الدالة تُرجع مؤشرًا إلى مصفوفة لم تعد موجودة. ولأن مصفوفة `fullName` مُعرّفة محليًا، فإنها تُدمر عند انتهاء الدالة. ستؤدي محاولة استخدام المؤشر إلى نتائج خاطئة وغير متوقعة.

يجب إرجاع مؤشر من دالة فقط إذا كان:

- مؤشرًا إلى عنصر مُمرّر إلى الدالة كوسيطة.
- مؤشرًا إلى جزء مُخصّص ديناميكيًا من الذاكرة.

على سبيل المثال، الدالة التالية مقبولة:

```
string *getFullName(string fullName[])
{
    cout << "Enter your first name: ";
    getline(cin, fullName[0]);
    cout << "Enter your middle name: ";
    getline(cin, fullName[1]);
    cout << "Enter your last name: ";
    getline(cin, fullName[2]);
    return fullName;
}
```

تقبل هذه الدالة مؤشرًا إلى موقع الذاكرة الذي ستُخزَّن فيه مُدخلات المستخدم. ولأن المؤشر يُشير إلى موقع ذاكرة كان صالحًا قبل استدعاء الدالة، فمن الآمن إرجاع مؤشر إلى الموقع نفسه. إليك دالة أخرى مقبولة:

```
string *getFullName()
{
    string *fullName = new string[3];

    cout << "Enter your first name: ";
    getline(cin, fullName[0]);
    cout << "Enter your middle name: ";
    getline(cin, fullName[1]);
    cout << "Enter your last name: ";
    getline(cin, fullName[2]);
    return fullName;
}
```

تستخدم هذه الدالة عامل التشغيل الجديد لتخصيص جزء من الذاكرة. ستبقى هذه الذاكرة مخصصة حتى استخدام عامل الحذف (delete) أو انتهاء البرنامج، لذا من الآمن إرجاع مؤشر إليها. يُظهر البرنامج 9-15 مثالاً آخر. يستخدم هذا البرنامج دالة getRandomNumbers للحصول على مؤشر لمصفوفة من الأرقام العشوائية. تقبل الدالة مُعاملاً صحيحاً يُمثل عدد الأرقام العشوائية في المصفوفة. تُخصص الدالة مصفوفة ديناميكيًا، وتُملأ بقيم عشوائية، ثم تُرجع مؤشرًا إلى المصفوفة.

Program 9-15

```
1 // This program demonstrates a function that returns a pointer.
2 #include <iostream>
3 #include <random>
4 using namespace std;
5
6 // Function prototype
7 int *getRandomNumbers(int);
8
9 int main()
10 {
```



```

11     int* numbers; // To point to the numbers
12
13     // Get an array of five random numbers.
14     numbers = getRandomNumbers(5);
15
16     // Display the numbers.
17     for (int count = 0; count < 5; count++)
18         cout << numbers[count] << endl;
19
20     // Free the memory.
21     delete [] numbers;
22     numbers = nullptr;
23     return 0;
24 }
25
26 //*****
27 // The getRandomNumbers function returns a pointer to an *
28 // array of random integers. The num parameter is the *
29 // number of numbers requested. *
30 //*****
31
32 int *getRandomNumbers(int num)
33 {
34     const int MIN = 0; // Minimum random number
35     const int MAX = 100; // Maximum random number
36     int* arr = nullptr; // Array to hold the numbers
37
38     // Random number engine and distribution object
39     random_device engine;
40     uniform_int_distribution<int> randInt(MIN, MAX);
41
42     // Return null if num is zero or negative.
43     if (num <= 0)
44         return nullptr;
45
46     // Dynamically allocate the array.
47     arr = new int[num];
48
49     // Populate the array with random numbers.
50     for (int count = 0; count < num; count++)
51         arr[count] = randInt(engine);
52
53     // Return a pointer to the array.
54     return arr;
55 }

```

Program Output

```

51
11
85
16
33

```

انتهت المحاضرة الثامنة